GPR-IQL Notebook Documentation

Overview

This notebook implements a hybrid offline reinforcement learning framework based on Implicit Q-Learning (IQL) integrated with Gaussian Process Regression (GPR) for stabilized value estimation and improved early-stage critic calibration.

The implementation targets continuous-control benchmarks from the D4RL MuJoCo suite and evaluates the effect of GPR-assisted supervision during offline policy optimization.

The notebook includes:

- MuJoCo environment setup
- Python dependency installation
- D4RL dataset loading
- Implicit Q-Learning (IQL) implementation
- Gaussian Process Regression (GPR) integration
- Offline training loops
- Critic and policy optimization
- Evaluation on MuJoCo benchmarks
- Logging and performance monitoring

System Requirements

Hardware

Recommended:

- CPU: Multi-core processor
- RAM: 16 GB or higher
- GPU: Optional but recommended for faster neural-network training
- Storage: At least 10 GB free space

Software

- Ubuntu/Linux environment recommended
- Python 3.10
- MuJoCo 2.1
- mujoco-py
- PyTorch
- D4RL benchmark suite

Notebook Structure

1. Environment Configuration

The notebook begins by configuring MuJoCo library paths:

```
os.environ['MUJOCO_PY_MUJOCO_PATH'] = '/root/.mujoco/mujoco210'
os.environ['LD_LIBRARY_PATH'] = '/root/.mujoco/mujoco210/bin:'
```

Purpose:

- Enables MuJoCo rendering and physics simulation
- Configures dynamic library loading
- Ensures compatibility with mujoco-py

---

2. Python Installation and Setup

The notebook installs Python 3.10 and associated development dependencies.

Key packages installed:

- python3.10
- python3-pip
- mujoco-py

---

3. Offline RL Framework Initialization

The implementation imports:

- PyTorch
- NumPy
- D4RL
- Gym
- Replay buffer utilities
- Optimization libraries

The notebook is based partially on:

- IQL-PyTorch implementation
- Implicit Q-Learning framework

Reference:

- https://arxiv.org/pdf/2110.06169.pdf

Core Methodology

Implicit Q-Learning (IQL)

IQL is an offline reinforcement learning algorithm that avoids direct maximization over out-of-distribution actions.

Core components include:

Value Function

The value network is optimized using expectile regression.

Critic Network

The critic estimates Q-values from offline transitions.

Policy Network

The policy is updated through advantage-weighted behavioral cloning.

Gaussian Process Regression (GPR) Integration

Motivation

Neural critics in offline RL often suffer from:

- unstable early optimization,
- noisy temporal-difference targets,
- distributional overestimation.

The notebook introduces Gaussian Process Regression as a stabilizing supervision mechanism.

GPR Objective

The Gaussian Process module generates smooth value approximations that are periodically used to guide critic learning.

Benefits include:

- smoother target estimation,
- improved critic calibration,
- reduced instability during early training,
- conservative value approximation.

Training Pipeline

Dataset Loading

The notebook loads offline datasets from D4RL environments.

Typical environments include:

- HalfCheetah-medium-v2
- Hopper-medium-v2
- Walker2d-medium-v2

Replay Buffer Construction

Offline trajectories are stored and sampled from replay memory.

Stored components:

- states
- actions
- rewards
- next states
- terminal flags

Critic Optimization

The critic network is optimized using temporal-difference targets combined with GPR-based supervision.

Training includes:

- Bellman backups
- target-network updates
- critic regularization
- GPR smoothing

Policy Optimization

Policy updates are advantage weighted.

The policy learns only from actions supported by the offline dataset.

Evaluation

The notebook evaluates:

- episodic return,

- critic stability,
- convergence behavior,
- training smoothness,
- offline RL performance.

Evaluation is performed periodically during training.

Important Hyperparameters

Typical parameters used in the notebook include:

Parameter	Description
gamma	Discount factor
tau	Expectile regression coefficient
beta	Advantage-weighting temperature
batch_size	Training batch size
learning_rate	Optimizer learning rate
gpr_subset_size	Number of samples used for GPR fitting

Expected Outputs

The notebook produces:

- training logs,
- evaluation scores,
- critic/value statistics,
- convergence plots,
- offline RL performance metrics.

Reproducibility Notes

To reproduce experiments:

1. Install MuJoCo and dependencies.
2. Configure environment variables.
3. Install required Python packages.
4. Download D4RL datasets.
5. Execute notebook cells sequentially.
6. Run evaluation cells after training convergence.

Potential Extensions

Future extensions may include:

- uncertainty-aware critic learning,
- sparse Gaussian Processes,
- transformer-based offline RL,
- ensemble critic stabilization,
- multi-task offline reinforcement learning,
- model-based planning integration.

References

1. Kostrikov et al., Implicit Q-Learning, 2021.
2. D4RL: Datasets for Deep Data-Driven Reinforcement Learning.
3. MuJoCo Physics Engine.
4. PyTorch Reinforcement Learning Frameworks.